

Neural Networks: From Logistic Regression to a Full Network

A Single Neuron as Logistic Regression 🧠

A single **artificial neuron** can be thought of as a simple logistic regression unit.

- **Job:** It takes one or more inputs, performs a computation, and produces a single output.
 - **Example (Demand Prediction):**
 - **Input (x):** Price of a T-shirt.
 - **Output (a):** Probability of it being a "top seller."
 - **Computation:** The standard logistic (sigmoid) function.
 - **New Terminology:** The output of a neuron, $f(x)$, is now referred to as its **activation**, denoted by the letter **a**. This term is borrowed from neuroscience.
-

Building a Neural Network

A neural network is formed by wiring many of these simple neurons together.

- **Example (Complex Demand Prediction):**
 - **Inputs ($\vec{x}$):** A vector of 4 features:
 1. Price (x_1)
 2. Shipping cost (x_2)
 3. Marketing (x_3)
 4. Material quality (x_4)
 - **Goal:** Predict if it's a "top seller" (y).

Layer-by-Layer Computation

1. **Input Layer:** This is not a computational layer; it's just the layer that holds the input feature vector $\vec{x}$.
2. **Hidden Layer:** This is a layer of neurons in the middle of the network.
 - Instead of feeding the raw inputs directly to the final prediction, we can first have a hidden layer compute *new, intermediate features*.
 - For example, this layer might have 3 neurons, each learning to compute a new concept:
 - Neuron 1 $\rightarrow$ (Affordability)
 - Neuron 2 $\rightarrow$ (Awareness)
 - Neuron 3 $\rightarrow$ (Perceived Quality)

- This layer takes the 4 input features ($\vec{x}$) and outputs a new vector of 3 activation values ($\vec{a}$).
- **Why is it "hidden"?** Because the training data provides the inputs (x) and the final output (y), but it does *not* provide the "correct" values for these intermediate features. Their values are "hidden" from the dataset.

3. **Output Layer:** This is the final layer that makes the prediction.

- It takes the activations from the hidden layer (the 3 new features) as its input.
- It performs a final logistic regression computation to output the probability of $y = 1$.

Standard Implementation: Fully Connected Layers

Manually deciding which inputs (e.g., *price*) go to which hidden neurons (e.g., *affordability*) is tedious.

- **Standard Practice:** In a neural network, layers are typically **fully connected**. This means **every neuron** in one layer receives its input from **every activation** in the previous layer.
- The network then *learns* the correct parameters (weights) to determine which inputs are important and which should be ignored for each neuron.

The Power of Neural Networks: Automatic Feature Engineering 🚀

- A neural network can be viewed as a **logistic regression model that automatically learns its own features**.
- In Course 1 (Polynomial Regression), we had to *manually* create new features (like x^2 , x^3 , or $x_1 \times x_2$).
- The **hidden layer** of a neural network performs this feature engineering automatically. It learns the most useful combination or transformation of the input features to make the final prediction easier and more accurate.

Network Architecture

- You are not limited to a single hidden layer. A network can have multiple hidden layers, where the output of one hidden layer becomes the input to the next.
- **Architecture:** The set of choices you make—such as the number of hidden layers and the number of neurons in each layer—is called the network's **architecture**.
- A network with multiple layers is sometimes called a **Multilayer Perceptron (MLP)**.